CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
DEPARTAMENTO DE COMPUTAÇÃO (DECOM)
LABORATÓRIO DE ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES I
Prof.ª Poliana Corrêa

PRÁTICA 6

Primeira versão do processador MIPS

Implementação dos estágios de busca, decodificação e execução da instrução

Vítor Albuquerque Dinelli

Belo Horizonte
Outubro de 2026

1 Introdução

Este relatório descreve a sexta prática da disciplina, cujo objetivo é dar o primeiro passo na construção de um processador MIPS de 32 bits. Nas práticas anteriores foram construídos blocos isolados (somador, ULA, banco de registradores, extensor de sinal e memória). Agora esses blocos são reunidos em um único projeto para que uma instrução seja buscada na memória, decodificada e executada.

O caminho de dados implementado corresponde à Figura 1 do roteiro: o PC endereça a memória de instruções, os campos da instrução selecionam os registradores a serem lidos no banco, e a ULA opera sobre o primeiro valor lido e sobre um segundo operando, que pode vir do registrador (readdata2) ou do imediato com sinal estendido. O resultado da ULA volta ao banco de registradores. Como ainda não existe a unidade de controle, os sinais RegDst, RegWrite, ALUSrc e ALUop são fornecidos manualmente, por chaves da placa ou, na simulação, pelo testbench.

Para comprovar o funcionamento, duas instruções foram armazenadas na memória: uma do tipo I (addi), que usa o sinal estendido como segundo operando, e outra do tipo R (add), que usa o valor lido do registrador. O projeto foi simulado no ModelSim e preparado para a placa DE10-Lite, com um módulo top level que mostra nos displays os valores do caminho de dados.

O relatório está organizado assim: a Seção 2 apresenta o projeto de cada bloco e as decisões tomadas; a Seção 3 apresenta as simulações; a Seção 4 descreve o roteiro de demonstração na placa; e a Seção 5 traz as conclusões. Os módulos das práticas anteriores (memoriaDados, ULA, parameterAdder, extensorSinal, bancoDeReg e decodificador) são apenas resumidos, por já terem sido documentados.

2 Projeto

2.1 Visão geral do caminho de dados

O módulo cpuMimps interliga todos os blocos da Figura 1 do roteiro. A Figura 1 abaixo mostra como ele foi montado, já indicando os campos da instrução usados em cada ponto (conforme a Figura 2 do roteiro) e os três estágios implementados.

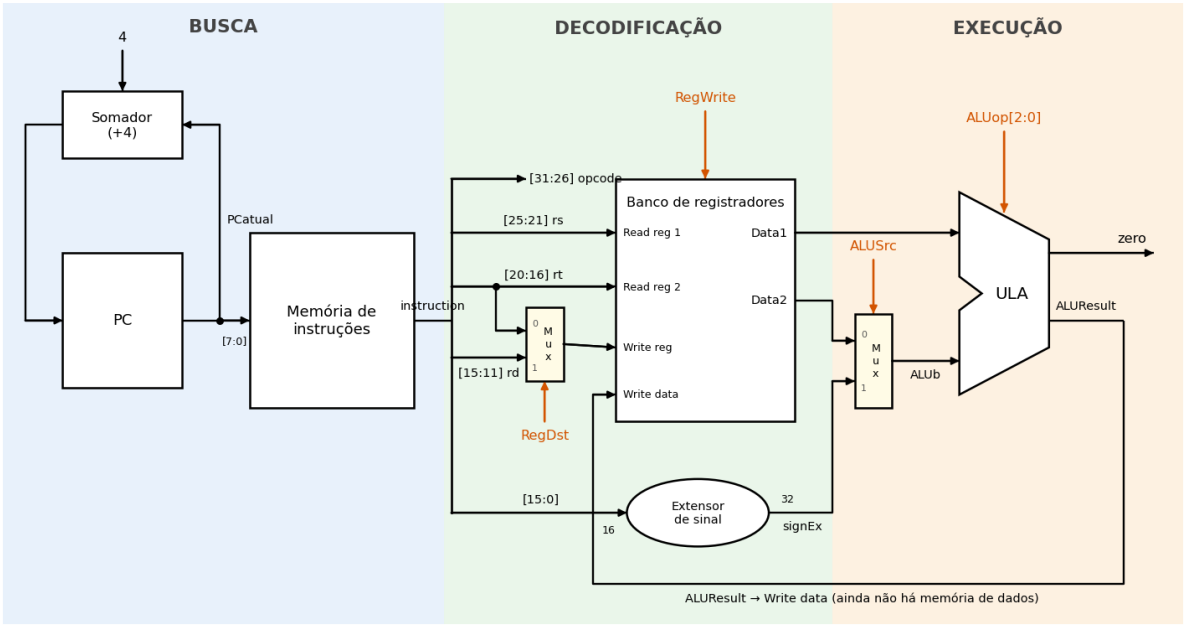


Figura 1 – Caminho de dados implementado no módulo cpuMimps

Em resumo, cada estágio faz o seguinte:

- **Busca:** o PC envia o endereço à memória de instruções e, em paralelo, o somador calcula PC + 4, valor que será gravado no PC na próxima borda de subida do clock.
- **Decodificação:** a instrução é dividida em campos. Os bits [25:21] (rs) e [20:16] (rt) endereçam as duas leituras do banco de registradores, o mux RegDst escolhe entre rt [20:16] e rd [15:11] como registrador de destino, e os bits [15:0] passam pelo extensor de sinal, virando um valor de 32 bits.
- **Execução:** o mux ALUSrc escolhe o segundo operando da ULA (readdata2 ou imediato estendido) e a ULA calcula o resultado de acordo com ALUop, gerando também o flag zero. O resultado é ligado à entrada Write data do banco, já que ainda não há memória de dados.

2.2 Blocos reaproveitados das práticas anteriores

Os módulos abaixo foram reutilizados sem alteração funcional. A tabela resume o papel de cada um neste projeto.

Tabela 1 – Módulos reaproveitados

Módulo	Função	Uso nesta prática
parameterAdder	Somador de N bits, construído com somadores completos encadeados	Soma PC + 4 (N = 32) e é usado dentro da ULA
ULA	Operações AND (000), OR (001), soma (010), subtração (110) e slt (111); gera os flags zero e overflow	Executa a operação da instrução (ALUop = 010)
bancoDeReg	32 registradores de 32 bits, leitura assíncrona de dois registradores e escrita síncrona	Lê rs e rt e grava o resultado em rt ou rd
extensorSinal	Estende de 16 para 32 bits replicando o bit de sinal	Transforma o imediato [15:0] em 32 bits

Módulo	Função	Uso nesta prática
memoriaDados	Memória endereçada por byte, com leitura assíncrona e escrita síncrona, montando palavras em little-endian	Usada internamente pela memória de instruções
decodificador	Converte um dígito de 4 bits em sinais de 7 segmentos (ativos em 0)	Mostra valores nos displays da placa

2.3 Registrador PC e somador PC + 4

O PC é um registrador de 32 bits com reset assíncrono (posedge clock ou posedge reset). Quando o reset está ativo ele volta para 0, caso contrário recebe o valor PCprox a cada borda de subida. O valor PCprox vem de uma instância do parameterAdder com N = 32, com a entrada b fixa em 32'd4 e cin em 0, pois cada instrução MIPS ocupa 4 bytes.

Foi mantido o PC com 32 bits, mesmo a memória sendo pequena, porque essa é a largura definida pela arquitetura MIPS e evita refazer o somador e o PC nas próximas práticas, quando desvios e saltos forem implementados. Apenas os 8 bits menos significativos são ligados à memória (PCatual[7:0]), que tem 256 bytes. O reset do PC recebe a condição (reset | ~ready), de modo que ele permanece em 0 enquanto a memória ainda está sendo carregada (Seção 2.4).

2.4 Memória de instruções (memoriaInstrucoes)

A memória de instruções reutiliza o módulo memoriaDados, instanciado com ADDR_SIZE = 8 (256 bytes) e DATA_BYTES = 4 (palavra de 32 bits). Como ela só precisa ser lida durante a execução, seus sinais foram travados: memRead = 1 e memWrite = ~ready.

A decisão mais importante foi como colocar as duas instruções nela. Gravá-las por um bloco initial com atrasos e pulsos de clock funcionaria apenas no simulador, pois o Quartus não sintetiza esse tipo de construção. Por isso foi criado um pequeno carregador feito de hardware: um contador de 2 bits (step) percorre as duas posições e, em cada borda de clock, uma lógica combinacional escolhe o endereço e a palavra a ser escrita usando a escrita normal da memoriaDados. Quando step chega em 2, o sinal ready sobe (ready = step[1]) e a memória passa a ser endereçada pelo PC.

O contador step não possui reset e começa em 0 pelo valor inicial da declaração. Assim a carga ocorre uma única vez, logo após a configuração da FPGA, e um reset posterior zera somente o PC, sem recarregar a memória. Esse comportamento é adequado porque o programa é fixo. A consequência prática é que os dois primeiros pulsos de clock depois de ligar a placa servem apenas para a carga. O sinal ready, no cpuMimps, também bloqueia a escrita no banco de registradores (RegWrite & ready), evitando que a instrução lida durante a carga altere algum registrador.

A memória é inicializada com zeros pelo initial da memoriaDados, de modo que endereços sem instrução devolvem 0x00000000 em vez de valores indefinidos na simulação.

2.5 Multiplexadores

Foi criado um multiplexador 2:1 parametrizável (mux) com entradas zero e um, seletor valor e saída s de N bits (s = valor ? um : zero). Ele é instanciado duas vezes: o mux0 (N = 5) escolhe o registrador de destino entre instruction[20:16] (RegDst = 0, instruções do tipo I) e instruction[15:11] (RegDst = 1, tipo R); o mux1 (N = 32) escolhe o segundo operando da ULA entre Data2 (ALUSrc = 0) e o imediato estendido (ALUSrc = 1).

2.6 Módulo cpuMimps

O cpuMimps instancia e conecta todos os blocos descritos acima, conforme a Figura 1. As entradas são o clock, o reset e os sinais de controle (RegDst, RegWrite, ALUSrc e ALUop[2:0]), fornecidos de fora do módulo. Além do flag zero, o módulo exporta como saídas os sinais internos necessários para a apresentação: PCatual, instruction, Data1, Data2, ALUb, ALUResult e ready. Isso permite que o top level mostre todo o caminho de dados seguido pelas instruções.

O dado a ser escrito no banco é o próprio ALUResult, pois nesta etapa não há memória de dados nem o mux de seleção entre resultado da ULA e dado lido da memória. A tabela a seguir resume como os bits da instrução são distribuídos.

Tabela 2 – Divisão dos bits da instrução na decodificação

Bits	Campo	Destino
`[31:26]`	opcode	Exibido no display (sem unidade de controle ainda)
`[25:21]`	rs	Read register 1 do banco de registradores
`[20:16]`	rt	Read register 2 e entrada 0 do mux RegDst
`[15:11]`	rd	Entrada 1 do mux RegDst
`[15:0]`	imediato	Extensor de sinal (32 bits) e entrada 1 do mux ALUSrc

2.7 Instruções armazenadas

As duas instruções escolhidas permitem exercitar os dois caminhos exigidos no roteiro: a primeira usa o sinal estendido como segundo operando da ULA e a segunda usa o valor de readdata2. Os registradores começam zerados, portanto a primeira instrução gera o valor que a segunda utiliza.

Tabela 3 – Instruções gravadas na memória

End.	Código (hexa)	Assembly	RegDst	ALUSrc	RegWrite	Resultado
0	`20080005`	addi $t0, $zero, 5	0	1	1	$t0 = 5
4	`01084820`	add $t1, $t0, $t0	1	0	1	$t1 = 10

Em ambos os casos ALUop = 010 (soma). Os campos da primeira instrução são: opcode 8, rs = 0, rt = 8 ($t0) e imediato 5. Os da segunda são: opcode 0, rs = 8, rt = 8, rd = 9 ($t1), shamt = 0 e funct = 0x20. Como a memória monta as palavras em little-endian, o carregador escreve os valores em hexadecimal normal e a memoriaDados se encarrega de separar os bytes.

2.8 Módulo top level para a placa

O módulo Top_cpu liga o processador aos recursos da DE10-Lite. As principais decisões foram:

- **Clock manual:** o clock do processador vem do botão KEY[0], para que cada instrução possa ser acompanhada. Como botões mecânicos geram ruído (bouncing), foi criado o módulo debounce, que sincroniza a entrada com o clock de 50 MHz e só altera a saída depois de cerca de 21 ms (contador de 20 bits) com o botão estável.
- **Reset:** KEY[1] (ativo em 0, invertido no código) ligado ao reset do cpuMimps.
- **Controles:** RegDst, RegWrite, ALUSrc e ALUop são fornecidos pelas chaves SW[0], SW[1], SW[2] e SW[5:3].
- **Visualização:** as chaves SW[8:6] escolhem o que aparece nos seis displays de 7 segmentos (Tabela 4), já que a placa não tem displays suficientes para mostrar tudo ao mesmo tempo.
- **Decimal nos displays:** o módulo bin2bcd converte o valor binário em cinco dígitos decimais (algoritmo double dabble) e cada dígito vai para o decodificador. Valores negativos são mostrados com o sinal "-" no HEX5 e o módulo em cinco dígitos; valores com mais de 99999 são limitados a esse número e acendem o LEDR[2].
- **LEDs:** LEDR[0] indica ready, LEDR[1] mostra o flag zero, LEDR[2] sinaliza valor fora do limite do display e LEDR[3] acompanha o clock já filtrado.

Tabela 4 – Seleção do que é mostrado nos displays (SW[8:6])

SW[8:6]	Valor mostrado	Formato
`000`	PC	decimal
`001`	opcode, instruction[31:26]	decimal
`010`	imediato, instruction[15:0]	decimal com sinal
`011`	Data1 (readdata1)	decimal com sinal
`100`	Data2 (readdata2)	decimal com sinal
`101`	ALUb (segundo operando da ULA)	decimal com sinal
`110`	ALUResult	decimal com sinal
`111`	rs | rt | rd (HEX5-4 | HEX3-2 | HEX1-0)	decimal

3 Simulações

3.1 Testbench

Foi criado o testbench tb_cpuMimps, que instancia o cpuMimps, gera um clock de 10 ns de período e aplica a sequência de estímulos abaixo. Os controles e o reset são alterados fora das bordas de subida (1 ns depois delas ou na borda de descida), pois os valores são capturados na borda de subida. A cada etapa, a task check compara os sinais com os valores esperados e imprime OK ou ERRO no console; no final, o testbench informa se todos os testes passaram. Os

conteúdos dos registradores $t0 e $t1 também são conferidos diretamente no banco por referência hierárquica, o que só é possível na simulação.

- Carga: dois clocks com reset ativo, após os quais ready deve ir para 1.
- Instrução 1 (addi): controles RegDst = 0, ALUSrc = 1, RegWrite = 1 e ALUop = 010; verifica PC = 0, Data1 = 0, ALUb = 5 e ALUResult = 5, e depois $t0 = 5 com PC = 4.
- Instrução 2 (add): troca para RegDst = 1 e ALUSrc = 0; verifica Data1 = 5, Data2 = 5, ALUb = 5 e ALUResult = 10, e depois $t1 = 10 com PC = 8.
- Fim do programa: com RegWrite = 0, o PC chega a um endereço sem instrução e a memória devolve 0.
- Reset: o PC volta a 0, ready continua em 1 e a instrução 0x20080005 continua na memória, mostrando que não há recarga.

3.2 Formas de onda

A Figura 2 mostra a simulação completa (86 ns). Os intervalos numerados correspondem às etapas descritas a seguir. As linhas amarelas tracejadas separam as etapas e o clock tem bordas de subida em 5, 15, 25 ns e assim por diante.

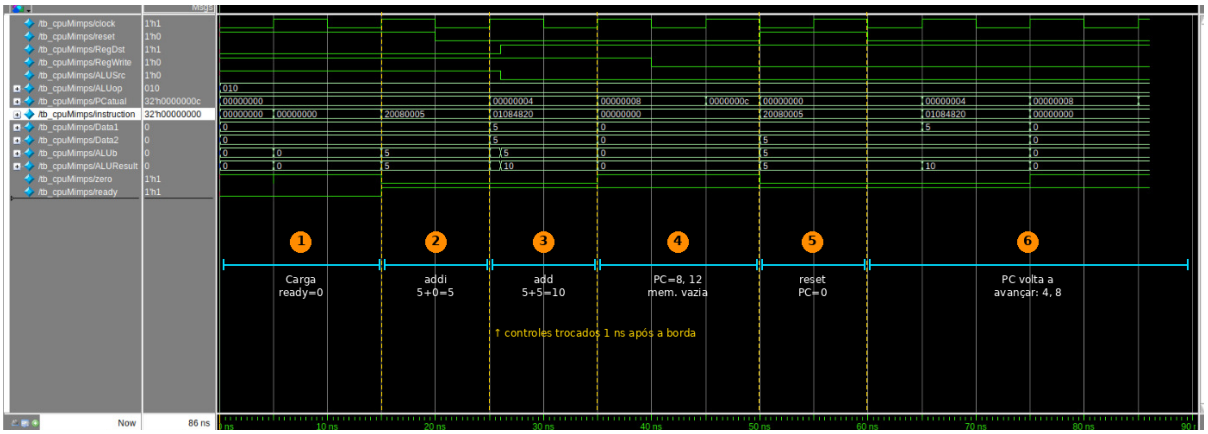


Figura 2 – Forma de onda da simulação do cpuMimps, com as etapas indicadas

- **Intervalo 1 (0 a 15 ns), carga:** reset = 1 e ready = 0. Nas bordas de 5 e 15 ns o carregador grava as duas instruções na memória. Logo após a segunda borda, ready vai para 1 e a instrução 0x20080005 aparece na saída.
- **Intervalo 2 (15 a 25 ns), addi:** com reset liberado em 20 ns, o PC vale 0. Com ALUSrc = 1, ALUb = 5 (imediato estendido), Data1 = 0 e ALUResult = 5. O flag zero fica em 0. Na borda de 25 ns o valor 5 é gravado em $t0.
- **Intervalo 3 (25 a 35 ns), add:** o PC passa a 4 e a instrução 0x01084820 aparece. Após a troca dos controles (RegDst = 1 e ALUSrc = 0), Data1 = 5, Data2 = 5, ALUb = 5 e ALUResult = 10. Na borda de 35 ns, $t1 recebe 10.
- **Intervalo 4 (35 a 50 ns):** o PC chega a 8 e depois a 12 (0xC), endereços sem instrução, e a leitura devolve 0x00000000. RegWrite é desligado em 40 ns para que nada seja escrito.

- **Intervalo 5 (50 a 60 ns), reset:** o PC volta imediatamente a 0 (reset assíncrono) e a instrução 0x20080005 volta a ser lida, sem nova carga. Nesse trecho os controles ainda são os da instrução 2, por isso ALUb e ALUResult mostram 5 (valor de $t0 somado a zero). Esse trecho serve apenas para mostrar o reset, e não como execução correta da instrução.
- **Intervalo 6 (60 a 90 ns):** com o reset liberado o PC volta a avançar (4, 8), mostrando que o processador continua funcionando.

A simulação evidencia os dois caminhos de dados exigidos no roteiro. Na primeira instrução o segundo operando da ULA vem do extensor de sinal (ALUb = 5, sem que Data2 seja usado), e na segunda vem do banco de registradores (ALUb = Data2 = 5). Também evidencia a escrita no banco de registradores, já que o 5 gravado por addi é lido como Data1 e Data2 na instrução seguinte, resultando em 10.

Há um detalhe visível na onda logo após a borda de 25 ns. Como os controles são trocados 1 ns depois da borda, durante esse 1 ns a instrução add já está na saída da memória, mas ALUSrc ainda vale 1. Nesse curto intervalo a ULA soma Data1 ao imediato estendido (0x4820, ou 18464, que são os bits [15:0] da instrução tipo R), produzindo um valor transitório que desaparece quando ALUSrc vai para 0. Em um processador com unidade de controle, os sinais mudariam junto com a instrução e esse transitório não existiria.

3.3 Saída do testbench

A Figura 3 mostra o final do console do ModelSim, com as verificações da etapa de reset e a mensagem de que todos os testes passaram, sem nenhum erro.



Figura 3 – Saída do testbench no console do ModelSim

4 Demonstração na placa

A síntese é feita com o módulo Top_cpu como entidade top-level, usando o arquivo de pinagem da DE10-Lite (dispositivo 10M50DAF484C7G). O roteiro de apresentação na placa é o seguinte, e os valores esperados em cada etapa estão na Tabela 5.

- Ao ligar a placa, LEDR[0] está apagado. Pressionar KEY[0] duas vezes carrega as instruções e acende LEDR[0] (ready).
- Para a instrução 1, deixar SW[0] = 0 (RegDst), SW[1] = 1 (RegWrite), SW[2] = 1 (ALUSrc) e ALUop = 010 (SW[5:3]). Percorrer as visões SW[8:6] para mostrar o caminho de dados.
- Pressionar KEY[0] para executar a instrução, e só depois alterar SW[0] = 1 e SW[2] = 0

para a instrução 2, repetindo a verificação.

- KEY[1] volta o PC a 0 para repetir a demonstração.

Tabela 5 – Valores esperados nos displays

Visão (SW[8:6])	addi (PC = 0)	add (PC = 4)
PC (000)	0	4
opcode (001)	8	0
imediato (010)	5	18464
Data1 (011)	0	5
Data2 (100)	0	5
ALUb (101)	5	5
ALUResult (110)	5	10
rs | rt | rd (111)	00 | 08 | 00	08 | 08 | 09

O imediato da instrução 2 é mostrado como 18464 porque, no formato R, os bits [15:0] correspondem aos campos rd, shamt e funct; esse valor não é usado, pois ALUSrc = 0 seleciona Data2.

5 Conclusão

Nesta prática foi implementada a primeira versão do caminho de dados do MIPS, interligando o PC, a memória de instruções, o banco de registradores, o extensor de sinal e a ULA, com os multiplexadores RegDst e ALUSrc. A simulação mostrou que as duas instruções armazenadas, addi e add, são buscadas, decodificadas e executadas corretamente, cada uma usando um dos dois caminhos possíveis para o segundo operando da ULA.

As principais decisões de projeto foram reaproveitar a memoriaDados como memória de instruções, carregada por um circuito sintetizável em vez de um initial com atrasos, e exportar os sinais internos do cpuMimps para que o top level mostre o caminho de dados nos displays. Como limitações desta versão, os sinais de controle são manuais, as instruções são fixas no código e os valores exibidos ficam limitados a cinco dígitos decimais.

Os próximos passos naturais são a inclusão da memória de dados (instruções lw e sw), o mux de seleção do dado a ser escrito no banco e a unidade de controle, que passará a gerar automaticamente os sinais hoje fornecidos pelas chaves.

Referências

PATTERSON, D. A.; HENNESSY, J. L. Organização e projeto de computadores: a interface hardware/software. 5. ed. Rio de Janeiro: Elsevier, 2017.

CORRÊA, P. Prática 6: Primeira versão do processador MIPS. Roteiro de laboratório. CEFET-MG, Departamento de Computação, 2026.